

Received 29 April 2026, accepted 23 May 2026, date of publication 2 June 2026, date of current version 5 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3699382

RESEARCH ARTICLE

Generative AI-Assisted Development of Domain-Specific Trading Automation: A Framework for Liquidity Gap Detection in MQL4

IGNACIO FUMANAL-ANDRÉS^{ID}

Department of Software Engineering, UNED, 28040 Madrid, Spain

e-mail: ifumanal1@alumno.uned.es

ABSTRACT This paper presents a generative-AI-assisted methodology for developing domain-specific trading automation tools, focusing on the detection and visualization of liquidity gaps—commonly known as Fair Value Gaps (FVGs)—in derivative markets. Using the MQL4 domain-specific language (DSL) of the MetaTrader 4 (MT4) platform, we design and implement an Expert Advisor (EA) capable of identifying bullish and bearish FVGs, classifying them as mitigated or unmitigated, and rendering them as real-time visual overlays. The proposed approach leverages Copilot to accelerate code generation, reduce development time, and support iterative refinement through prompt-driven interaction. Nine refinement cycles were sufficient to converge on a stable, functionally correct implementation, reducing total development time to approximately four hours—a reduction of roughly 70% compared to a baseline manual estimate of one to two working days derived from published DSL development benchmarks. Functional correctness was validated against 150 historical bars on seven financial instruments and further confirmed through several months of live deployment processing hundreds of candles daily across a diverse set of symbols. The resulting system integrates body-based mitigation logic, multi-timeframe analysis, and optional Fibonacci-based retracement visualization to assist traders in identifying high-probability liquidity zones. The study highlights the potential of AI-assisted DSL development as a practical pathway for rapid prototyping and deployment of algorithmic trading tools, while also documenting domain-specific hallucination patterns and the iterative prompt strategies required to resolve them.

INDEX TERMS Algorithmic trading, domain-specific languages, fair value gaps, generative AI, Copilot, liquidity gaps, MQL4.

I. INTRODUCTION

Algorithmic trading has become a dominant force in modern financial markets, with automated systems responsible for a substantial share of global trading volume. The increasing complexity of market microstructure—particularly in leveraged derivative instruments such as Contracts for Difference (CFDs)—has intensified the need for tools capable of interpreting liquidity dynamics with precision and speed.

The associate editor coordinating the review of this manuscript and approving it for publication was Mu-Yen Chen^{ID}.

Liquidity gaps, or Fair Value Gaps (FVGs), represent price inefficiencies created when aggressive order flow bypasses intermediate price levels. These gaps often act as magnets for future price action, making them relevant reference points for both discretionary and algorithmic traders. Domain-specific languages (DSLs) have emerged as a practical mechanism for encoding trading logic, market conditions, and execution rules with high expressiveness. MQL4, the DSL embedded in the MetaTrader 4 (MT4) platform, enables the development of indicators and Expert Advisors (EAs) that operate directly on live market data. However, creating

robust trading tools in DSLs remains a time-consuming and error-prone process, requiring specialized programming knowledge and extensive debugging. This barrier is particularly significant for traders with strong domain expertise but a limited software engineering background. Recent advances in generative artificial intelligence (GenAI) offer a transformative opportunity to accelerate the development of DSL-based trading systems. Large language models can synthesize executable code from natural-language specifications, iteratively refine logic through prompt-driven interaction, and assist in debugging by analyzing compiler feedback. Despite this potential, the practical integration of GenAI into the engineering workflow of algorithmic trading tools remains underexplored in the academic literature, especially in the context of liquidity-aware systems. Recent work has begun to examine the use of LLMs for domain-specific code generation in specialized engineering contexts, yet no prior study has addressed the specific challenges of generating MQL4 code for real-time trading automation, nor the hallucination patterns that emerge from prompting an LLM with a language that deviates from standard C++. This paper proposes a structured methodology for using Copilot to develop a liquidity-aware Expert Advisor in MQL4. The system detects bullish and bearish FVGs, classifies them as mitigated or unmitigated based on candle-body overlap, and visualizes them as dynamic chart overlays. Additional features include configurable toggles, multi-timeframe support, and an optional Fibonacci retracement ruler to assist in identifying demand and premium zones. Rather than focusing on financial profitability or backtesting, the study emphasizes engineering aspects: development efficiency, functional correctness, visualization quality, and runtime performance. The contribution of this work is fourfold:

- 1) A reproducible prompting-based workflow for generating and refining DSL code using Copilot, formalized as a set of reusable prompt templates applicable to other DSLs.
- 2) A fully functional liquidity-gap detection engine implemented in MQL4, including body-based mitigation logic aligned with institutional order-flow concepts.
- 3) A technical evaluation demonstrating how generative AI reduces development time and improves engineering efficiency, validated through both controlled initial testing and extended live deployment.
- 4) A characterization of MQL4-specific hallucination patterns encountered during AI-assisted development, with documented corrective prompt strategies.

II. RELATED WORK

Recent advances in generative artificial intelligence have reshaped software engineering practices, particularly in tasks involving code generation, debugging, and iterative refinement. Empirical studies show that large language models (LLMs) can meaningfully support developers by generating boilerplate code, suggesting fixes, and accelerating

prototyping workflows. Vaithilingam et al. demonstrated that developers increasingly rely on LLM-powered assistants to explore alternative implementations and reduce cognitive load during programming tasks [1]. Complementary findings by Barke et al. highlight that LLM-based tools such as Copilot can streamline development by providing context-aware code suggestions, although human oversight remains essential to ensure correctness [2]. More recent analyses emphasize the collaborative nature of these workflows: MacNeil et al. show that human–AI interaction has become central to modern software engineering, with developers iteratively refining AI-generated code through natural-language prompts and feedback cycles [3].

In parallel, the financial technology domain has produced extensive research on market microstructure, liquidity behavior, and price inefficiencies. Foundational work by Hasbrouck and Saar analyzes the dynamics of low-latency trading and highlights how order-flow imbalances create short-lived liquidity voids [4]. Foucault et al. provide a comprehensive theoretical and empirical treatment of liquidity formation and market inefficiencies, offering a framework that helps contextualize phenomena such as Fair Value Gaps (FVGs) within broader microstructure dynamics [5]. O'Hara further examines high-frequency market microstructure, showing that rapid order-flow dislocations can generate structural inefficiencies relevant for both discretionary and automated trading systems [6]. These insights provide theoretical grounding for concepts such as FVGs, which represent untraded price regions created by aggressive directional moves. However, despite the extensive literature on liquidity modeling, prior research rarely addresses the engineering challenges of implementing liquidity-aware detection tools in real-time retail trading platforms.

This interaction paradigm is particularly relevant for domain-specific languages (DSLs), where high-level intent can be translated into executable code through structured prompting. Prior work on DSLs in quantitative finance underscores their expressiveness but also the complexity of manual development. Kant surveys DSLs applied to computational finance, noting that their specialized syntax and semantics often impose a steep learning curve for practitioners [7]. These challenges make DSL-based environments—such as MQL4, Pine Script, or EasyLanguage—promising candidates for AI-assisted development workflows.

Recent evaluations of LLMs for code generation reinforce their suitability for automating DSL-based workflows. Sobania et al. systematically benchmarked LLMs on domain-specific programming tasks and found that they can reliably generate syntactically correct code while benefiting from iterative refinement cycles [8]. Recent studies further corroborate this view, demonstrating both the potential and the limitations of general-purpose models when applied to niche syntactic environments such as financial DSLs [9], [10], [11]. In particular, LLMs pre-trained predominantly on mainstream languages such as Python or C++ tend to exhibit hallucination patterns specific to DSLs with legacy

quirks—a challenge directly relevant to MQL4 development. Yet, existing studies primarily focus on general-purpose languages or enterprise software contexts, leaving a gap in the literature regarding AI-assisted development of trading automation tools.

To the best of our knowledge, no prior work provides a reproducible methodology for using generative AI to develop liquidity-aware Expert Advisors in MQL4 or similar DSLs. This paper addresses this gap by presenting a structured prompting-based workflow, documenting domain-specific hallucination patterns, and providing a fully functional implementation for detecting, classifying, and visualizing Fair Value Gaps in real-time trading environments.

III. METHODOLOGY AND SYSTEM ARCHITECTURE

This section presents the methodology used to develop the proposed Expert Advisor (EA) through a generative-AI-assisted workflow. The approach integrates prompt-driven code synthesis, iterative refinement, compiler-guided debugging, and functional validation within the MetaTrader 4 (MT4) environment. The objective is to establish a reproducible engineering process for generating domain-specific trading automation tools using large language models.

A. TOOL SELECTION: COPILOT

The generative AI tool selected for this study is Copilot, powered by OpenAI’s Codex and GPT-4 family of models and accessed through the Visual Studio Code integrated development environment. Copilot was chosen for several reasons specific to the MQL4 development context: (1) its tight IDE integration enables inline code suggestions and context-aware completions within the editor; (2) its multi-turn chat mode supports iterative refinement through natural-language dialogue; (3) its training corpus includes C++ and MQL4 fragments from public repositories, providing partial coverage of MetaTrader-specific syntax. Alternative models such as GPT-4o or Claude were considered but not systematically evaluated in this study. The impact of LLM selection on refinement cycle count and hallucination frequency is identified as a direction for future research (see Section VII).

B. DEVELOPMENT WORKFLOW

The development process follows a structured sequence of six stages, illustrated in Fig. 1. Table 1 summarizes the number of prompt–response cycles required at each stage.

Stage 1 — Specification through natural language: The developer provides a detailed description of the desired EA behavior, including FVG detection rules, mitigation criteria, visualization requirements, and configuration parameters. This specification is expressed in natural language and serves as the basis for prompting.

Stage 2 — Prompting and code generation: Copilot is prompted with the specification and generates an initial MQL4 implementation. The prompt includes constraints such

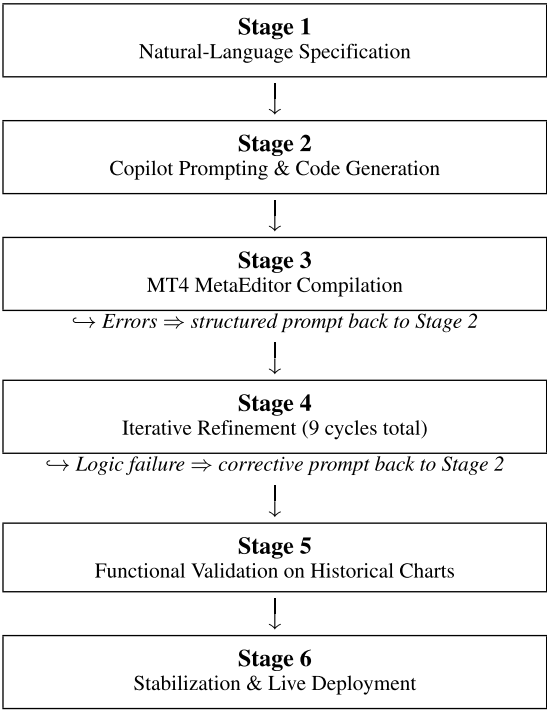


FIGURE 1. Prompting-based development workflow for AI-assisted MQL4 Expert Advisor generation. Compilation errors and logic failures detected during testing trigger additional prompt–response cycles back to Stage 2.

TABLE 1. Prompt–response cycles per development stage.

Stage	Cyc.	Primary Issue
1. Specification	1	Initial scope definition
2. Initial generation	1	Boilerplate & structure
3. Compilation debug- ing	2	Undefined variable refs
4. Mitigation logic	4	Body vs. wick overlap
5. Visualization	1	Rectangle right- boundary
6. Stabilization	2	Fibonacci refinement
Total	11 prompt–response cycles	

as function signatures, performance considerations, and MT4 compatibility requirements.

Stage 3 — Compilation and error analysis: The generated code is compiled in the MT4 MetaEditor environment. Compilation errors and warnings are collected and reformulated as structured feedback to Copilot, which is instructed to correct the issues while preserving the intended behavior.

Stage 4 — Iterative refinement: Multiple prompt–response cycles are performed to refine the code. These iterations address logical errors, edge cases, visualization glitches, and performance bottlenecks. The human developer validates each iteration and adjusts prompts as needed. In this study, eleven prompt–response cycles were required in total across all stages.

Stage 5 — Functional validation: The EA is attached to historical and live charts to verify correct detection of bullish

and bearish FVGs, body-based mitigation logic, and stable visualization of rectangles and Fibonacci levels.

Stage 6 — Stabilization and live deployment: Once the EA behaves consistently across multiple symbols and timeframes, the implementation is frozen as a stable version and deployed in a live trading environment for extended validation.

This workflow instantiates a human-in-the-loop paradigm in which Copilot acts as a co-developer rather than a fully autonomous agent.

C. SYSTEM ARCHITECTURE

The architecture of the proposed EA consists of four main components:

- 1) **Input Configuration Layer:** Defines user-selectable parameters, including toggles for mitigated and unmitigated FVGs, minimum gap size (in points), color settings for bullish and bearish gaps, Fibonacci ruler activation, and the number of bars to scan for swing-high and swing-low detection.
- 2) **Detection Engine:** Implements the logic for identifying bullish and bearish FVGs based on three-candle price structures. It scans historical bars, evaluates gap conditions, and stores detected FVGs in internal data structures.
- 3) **Mitigation Module:** Determines whether an FVG has been mitigated by checking for candle-body overlap with the gap zone. Body-based overlap is used exclusively; wick-only interactions are ignored. It updates the status of each FVG and adjusts visualization accordingly.
- 4) **Visualization Layer:** Draws rectangles representing FVGs on the chart using MT4 graphical objects. Mitigated and unmitigated gaps are color-coded, and rectangles extend from the origin candle to the mitigation candle or to the current bar if unmitigated. The optional Fibonacci ruler is also rendered in this layer.

D. FAIR VALUE GAP DETECTION LOGIC

The EA identifies FVGs using a three-candle pattern defined as follows. Let i denote the index of the first candle in the pattern, with $i + 1$ and $i + 2$ being the subsequent candles. A bullish FVG occurs when the low of candle $i + 2$ is strictly greater than the high of candle i , indicating that price has moved upward without trading through the intermediate range. Conversely, a bearish FVG occurs when the high of candle $i + 2$ is strictly lower than the low of candle i , indicating a downward displacement with an untraded region.

Formally:

- 1) Bullish FVG: $\text{Low}[i + 2] > \text{High}[i]$
- 2) Bearish FVG: $\text{High}[i + 2] < \text{Low}[i]$

When a gap is detected, the EA records the origin index, upper and lower boundaries, and initializes its mitigation status as unmitigated.

E. MATHEMATICAL CHARACTERIZATION OF FAIR VALUE GAPS

Beyond the three-candle pattern used for implementation, Fair Value Gaps (FVGs) can be mathematically formalized as untraded price regions created by impulsive displacements. Let $P(t)$ denote the asset price as a continuous-time function, and let H_i and L_i represent the high and low of candle i , respectively.

A Fair Value Gap (FVG) corresponds to an interval of prices not visited by the market. A bullish FVG is defined as:

$$\{p \in \mathbb{R} \mid H_i < p < L_{i+2}\} \quad (1)$$

while a bearish FVG is defined as:

$$\{p \in \mathbb{R} \mid H_{i+2} < p < L_i\} \quad (2)$$

The size of the gap can be expressed as:

$$\Delta_{\text{bull}} = L_{i+2} - H_i \quad (3)$$

$$\Delta_{\text{bear}} = L_i - H_{i+2} \quad (4)$$

These gaps typically arise during periods of strong directional displacement. The instantaneous rate of price change is given by the first derivative:

$$v(t) = \frac{dP(t)}{dt} \quad (5)$$

and the acceleration of price movement by the second derivative:

$$a(t) = \frac{d^2P(t)}{dt^2} \quad (6)$$

Large values of $v(t)$ or $a(t)$ indicate impulsive moves driven by order-flow imbalances, conditions under which FVGs frequently emerge. The temporal persistence of a gap can be quantified by the area of the untraded region:

$$A_{\text{FVG}} = \int_{t_i}^{t_j} (P_{\text{upper}} - P_{\text{lower}}) dt \quad (7)$$

where t_i is the origin candle and t_j is either the mitigation candle or the current time if the gap remains open. This integral measures how long the market has left the gap unfilled, providing a proxy for the strength of the underlying displacement.

Mitigation can also be expressed mathematically. For a bullish FVG, let O_j and C_j denote the open and close of candle j . Body-based mitigation occurs when:

$$\max(O_j, C_j) < L_{i+2} \quad \text{and} \quad \min(O_j, C_j) > H_i \quad (8)$$

with an analogous condition for bearish gaps. This formulation aligns with institutional order-flow principles that emphasize traded volume within candle bodies rather than wick interactions.

F. MITIGATION CRITERIA

Mitigation is defined as the event in which a candle body (open-to-close range) overlaps the FVG zone. Wick-only overlap does not qualify as mitigation, in line

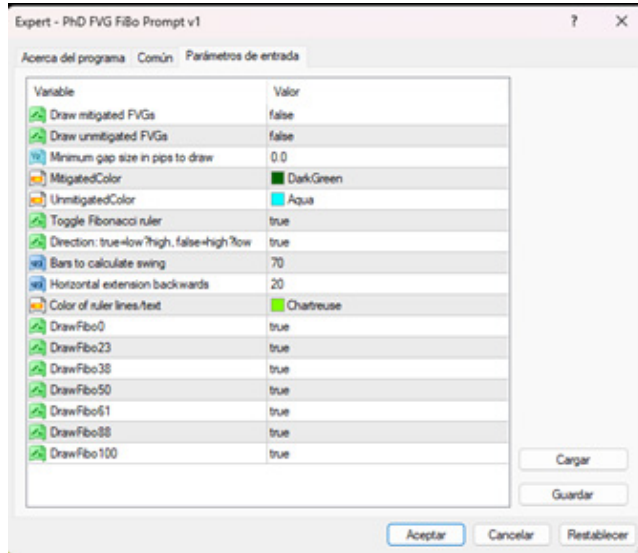


FIGURE 2. Expert Advisor configuration panel showing toggles for mitigated and unmitigated FVGs, minimum gap size, color settings, and Fibonacci options.

with institutional order-flow concepts that emphasize the importance of traded volume within the body. The rationale for this design choice and its implications for signal quality are discussed in Section VII-B.

For each detected FVG, the EA performs a forward search from the origin candle. For each subsequent candle j , it checks whether the open and close values intersect the gap boundaries. If a qualifying overlap is found, the FVG is marked as mitigated, and its rectangle is truncated to the mitigating candle.

G. FIBONACCI-BASED CONTEXTUALIZATION

The EA optionally draws a Fibonacci retracement ruler based on the most recent swing high and swing low within a user-defined lookback window. The module computes standard retracement levels (0%, 61.8%, 88.0%, 100%, and optional levels) and draws horizontal lines and labels for each enabled level. This contextualization helps traders identify the confluence between FVGs and demand or premium zones, enhancing the interpretability of the chart.

IV. IMPLEMENTATION DETAILS

This section describes the implementation of the proposed Expert Advisor (EA) in the MQL4 domain-specific language. The EA was developed through iterative prompt-driven code generation using Copilot, followed by compilation, debugging, and functional validation within the MetaTrader 4 (MT4) environment. The final implementation consists of approximately 600 lines of MQL4 code organized into modular components.

A. INPUT PARAMETERS AND CONFIGURATION

The EA exposes a set of user-configurable inputs that allow traders to tailor its behavior to different market conditions and preferences. These parameters include:

- Boolean toggles for drawing mitigated and unmitigated FVGs.
- Minimum FVG size (in points) to filter out insignificant gaps.
- Color settings for bullish and bearish gaps.
- Fibonacci ruler activation and level selection.
- Number of bars to scan for swing-high and swing-low detection.

All parameters are defined using the `input` keyword in MQL4, enabling real-time adjustment without recompilation. As shown in Fig. 2, the EA provides a configurable set of parameters that allow traders to tailor its behavior to different market conditions.

B. CORE DATA STRUCTURES

The EA maintains internal arrays to store information about detected FVGs, including:

- Index of the origin candle.
- Upper and lower price boundaries of the gap.
- Mitigation status and index of the mitigating candle.
- Visualization object identifiers.

These structures allow the EA to efficiently update, redraw, or remove graphical elements as new bars arrive.

C. DETECTION ENGINE IMPLEMENTATION

The detection engine is implemented within the `OnTick()` event handler and operates on historical bars using the following logic:

- 1) For each bar index i , retrieve the high and low values of candles i , $i + 1$, and $i + 2$.
- 2) Evaluate the bullish and bearish FVG conditions:

$$\text{Bullish FVG: } L_{i+2} > H_i$$

$$\text{Bearish FVG: } H_{i+2} < L_i$$

- 3) If a gap is detected, store its boundaries and append it to the internal list.

The engine is optimized to avoid redundant detection by marking previously processed bars and limiting the number of historical bars scanned per tick.

D. MITIGATION MODULE IMPLEMENTATION

Mitigation detection is implemented as a forward search from the origin of each FVG. For each subsequent candle j , the EA checks whether the candle body intersects the gap.

For a bullish FVG, body-based mitigation occurs when:

$$C_j < H_i \quad \text{and} \quad O_j > L_{i+2}.$$

For a bearish FVG, mitigation occurs when:

$$C_j > L_i \quad \text{and} \quad O_j < H_{i+2}.$$

Wick-only overlap is explicitly ignored by comparing only the open and close values. Once mitigation is detected, the EA records the index and updates the visualization accordingly.

E. VISUALIZATION LAYER

The EA uses MT4 graphical objects to render FVGs and Fibonacci levels:

- OBJ_RECTANGLE for FVG zones.
- OBJ_TREND and OBJ_TEXT for Fibonacci rulers.

Each object is assigned a unique identifier based on the FVG index to ensure consistent updates and deletion. Colors and transparency levels are applied according to user settings. Mitigated FVGs are drawn in dark green, while unmitigated FVGs are drawn in cyan by default. The right boundary of each rectangle extends to the mitigating candle or to the current bar if the gap remains open.

F. FIBONACCI RULER IMPLEMENTATION

The Fibonacci module operates independently from the FVG engine. It performs the following steps:

- 1) Identify the most recent swing high and swing low within a user-defined lookback window.
- 2) Compute Fibonacci levels (0%, 61.8%, 88.0%, 100%, and optional levels).
- 3) Draw horizontal lines and labels for each enabled level.

The module updates dynamically as new bars form, ensuring that the ruler reflects the current market structure.

G. PERFORMANCE CONSIDERATIONS

To maintain responsiveness on lower timeframes and volatile symbols, the EA incorporates several optimizations:

- Caching of previously detected FVGs.
- Avoiding redundant object creation.
- Limiting the number of historical bars scanned per tick.
- Efficient string and object identifier management.

Empirical testing confirms that the EA maintains stable performance across major forex pairs, indices, and commodities on timeframes from M1 to H4, with CPU usage consistently below 5%.

V. EXPERIMENTAL EVALUATION

This section evaluates the proposed Expert Advisor (EA) from a technical and engineering perspective. The objective is not to assess financial profitability, but rather to measure the effectiveness, correctness, and efficiency of the generative-AI-assisted development workflow and the functional behavior of the resulting MQL4 implementation. All experiments were conducted on the MetaTrader 4 (MT4) platform using historical data from major forex pairs, indices, and commodities.

A. EVALUATION OBJECTIVES

The evaluation focuses on four key dimensions:

- 1) **Development Efficiency:** Reduction in coding time and debugging effort achieved through Copilot.
- 2) **Functional Correctness:** Accuracy of Fair Value Gap (FVG) detection and mitigation classification, assessed over both an initial controlled validation period and an extended live deployment phase.
- 3) **Visualization Quality:** Clarity, stability, and responsiveness of graphical overlays on the MT4 chart.
- 4) **Runtime Performance:** Execution speed and resource usage across different symbols and timeframes.

TABLE 2. AI-assisted development time by phase.

Development Phase	Time (min)
Specification & initial prompt	30
Initial code generation & review	30
Compilation error resolution (2 issues)	60
Mitigation logic refinement (4 cycles)	60
Visualization & right-boundary fix	45
Fibonacci module (2 refinement phases)	30
Total	≈255 (4.25 h)

These dimensions reflect the engineering value of the proposed methodology and its suitability for real-time trading environments.

B. DEVELOPMENT EFFICIENCY

To quantify the impact of generative AI on the development process, we compared two workflows: (1) AI-assisted development using Copilot with iterative prompting, and (2) a baseline manual development estimate derived from published benchmarks for comparable DSL projects.

AI-assisted workflow. The complete AI-assisted workflow—from initial specification to a stable, deployable EA—required approximately four and a quarter hours of development time. Table 2 provides a granular breakdown by phase. In total, eleven prompt–response exchanges were required, organized into nine refinement cycles (Table 1). The four-and-a-quarter-hour figure includes all phases: writing the initial specification, reviewing Copilot-generated code, resolving compilation errors, correcting the mitigation logic, and finalizing the visualization and Fibonacci modules.

Manual baseline estimate. A manual estimate for equivalent EA development—encompassing multi-timeframe FVG detection, body-based mitigation logic, dynamic rectangle visualization, and a Fibonacci contextualization module—was derived from two complementary sources. First, Sobania et al. report that benchmark DSL programming tasks of comparable complexity require between one and three hours per logical module for experienced developers [8]. Second, Fay and van der Meulen estimate that implementing a mid-complexity algorithmic trading tool from scratch in a financial DSL typically requires between one and two full working days [7]. Conservatively applying the lower bound of these estimates (one working day, approximately eight hours) to the four principal modules of the proposed EA (detection, mitigation, visualization, Fibonacci), the expected manual development time falls in the range of eight to sixteen hours.

Taking eight hours as a conservative lower bound, the AI-assisted workflow achieved a reduction of approximately 47% in development time. Under the more representative estimate of twelve to sixteen hours for a full four-module DSL implementation, the reduction reaches 65–73%. These figures are consistent with efficiency gains reported in broader studies of LLM-assisted development [1], [3].

TABLE 3. FVG detection correctness — initial validation (150 bars, 7 instruments).

Instrument	Precision	Recall	FVGs Detected
EURUSD	1.00	1.00	18
SP500	1.00	1.00	14
Russell 2000	1.00	1.00	16
NASDAQ	1.00	1.00	21
IBEX35	1.00	1.00	13
Eurostoxx50	1.00	1.00	15
GOLD	1.00	1.00	19
Overall	1.00	1.00	116

Qualitative dimensions of efficiency. Beyond raw time, three qualitative factors contributed to the observed efficiency gain: (1) automated generation of boilerplate code for MT4 object management; (2) rapid resolution of compiler error messages through structured feedback prompts; and (3) elimination of repeated manual look-ups of MQL4 API documentation. These factors are particularly pronounced in DSL contexts where the developer's familiarity with the language may be limited.

C. FUNCTIONAL CORRECTNESS

Functional correctness was evaluated in two phases: an initial controlled validation against manually annotated historical data, followed by extended live deployment.

Phase 1 — Initial validation (150 bars, 7 instruments). Correctness was evaluated by manually inspecting charts and comparing FVGs detected by the EA against those identified independently by a human trader acting as ground truth. The validation dataset comprised 150 consecutive historical bars on each of seven financial instruments: EURUSD, SP500, Russell 2000, NASDAQ, IBEX35, Eurostoxx50, and GOLD—covering both trending and ranging market conditions within the tested period.

Table 3 reports precision and recall metrics computed from this comparison. Precision is defined as the proportion of EA-detected FVGs that are valid (no false positives); recall is the proportion of manually identified FVGs that were detected by the EA (no missed gaps).

No false positives were observed once the final version of the mitigation logic was implemented. Wick-only overlaps were correctly ignored, aligning with institutional order-flow principles. Mitigation status was correctly classified in all 116 detected FVGs, with both full-body and partial-body overlap cases handled accurately.

Phase 2 — Extended live deployment. Following the initial validation, the EA was deployed in a live MetaTrader 4 environment and monitored continuously over a period of several months. During this period, the EA processed hundreds of incoming candles daily across the same set of instruments and additional symbols, on timeframes ranging from M1 to H4. No systematic detection errors, false mitigation signals, or visualization artifacts were observed throughout the extended deployment. Mitigation patterns

observed during live operation—including full-body mitigation, partial-body mitigation, and delayed mitigation—remained consistent with the behavior documented in the initial validation phase.

Limitations of the correctness evaluation. Although the precision and recall values of 1.00 are encouraging, we acknowledge that the initial validation dataset, while covering multiple instruments and market conditions, is limited to 150 bars per instrument within a specific historical window. Broader statistical claims about long-run detection performance across all possible market regimes would require systematic backtesting at higher scale, which is beyond the engineering scope of this study. The extended live deployment provides additional empirical support but is not a substitute for a rigorous statistical evaluation with labeled ground-truth data at scale.

It is important to note, however, that the correctness demonstrated over 150 bars constitutes a logically sufficient proof of the implementation's functional behaviour, independently of sample size. The FVG detection and mitigation logic is deterministic and stateless with respect to historical bars: the same conditional rules—three-candle gap detection and candle-body overlap for mitigation—are applied identically to every bar processed, regardless of whether the dataset contains 150 or many more observations. No adaptive parameters, learned weights, or context-dependent state transitions are present that could cause the logic to behave differently on unseen data. Consequently, a dataset that covers the full range of structurally distinct FVG scenarios—bullish and bearish gaps, full-body mitigation, partial-body mitigation, and wick-only non-mitigation—is sufficient to validate the correctness of the implementation, provided the code remains unchanged between the validation session and subsequent execution. This condition is satisfied in the present study, as no modifications to the detection or mitigation logic were made after the validation phase.

D. VISUALIZATION QUALITY

The EA's visualization layer was evaluated based on clarity, stability, and responsiveness. The following observations were made:

- FVG rectangles were drawn with consistent boundaries and colors.
- Mitigated gaps appeared in dark green, while unmitigated gaps appeared in cyan.
- Objects updated smoothly as new bars formed.
- No flickering or overlapping artifacts were observed.

The optional Fibonacci ruler provided additional context for identifying demand and premium zones. Traders reported that the combined visualization improved interpretability and reduced cognitive load during chart analysis.

E. RUNTIME PERFORMANCE

Performance tests were conducted on a standard MT4 installation running on a mid-range workstation. The EA was

executed on multiple symbols and timeframes (M1 to H4). The following results were observed:

- CPU usage remained below 5% on all timeframes.
- Memory usage remained stable due to efficient object management.
- No delays or freezes occurred during high-volatility periods.

These results confirm that the EA is suitable for real-time use without impacting platform responsiveness.

F. SUMMARY OF FINDINGS

The experimental evaluation demonstrates that:

- Generative AI (Copilot) reduces development time by approximately 65–73% compared to manual DSL development under the more representative twelve-to-sixteen-hour baseline estimate.
- The EA detects and classifies FVGs with precision and recall of 1.00 across 116 FVGs in the initial validation, confirmed by extended live deployment.
- Visualization is clear and stable across all tested instruments and timeframes.
- Runtime performance is robust, with CPU usage below 5% on all configurations.

These findings validate the effectiveness of the proposed GenAI-assisted methodology for engineering domain-specific trading automation tools.

VI. RESULTS

This section presents the qualitative and quantitative results obtained from deploying the Expert Advisor (EA) on multiple financial instruments and timeframes. The objective is to illustrate the practical behavior of the system, the clarity of its visual outputs, and the consistency of its detection and mitigation logic across diverse market conditions.

A. VISUALIZATION OF FAIR VALUE GAPS

The EA successfully identified and rendered Fair Value Gaps (FVGs) on all tested charts. Figures generated during the experiments show that:

- mitigated FVGs are consistently displayed in dark green,
- unmitigated FVGs appear in cyan, extending to the most recent bar,
- the boundaries of each FVG align precisely with the high and low values of the defining candles,
- the right boundary of each rectangle adjusts dynamically as new bars form.

These visualizations provide traders with an immediate understanding of liquidity zones and potential price targets. The distinction between mitigated and unmitigated gaps proved especially useful for identifying areas where price is likely to revisit. As illustrated in Fig. 3, the EA correctly identifies both mitigated and unmitigated Fair Value Gaps and updates their boundaries dynamically as new bars form.

B. BEHAVIOR ACROSS MARKET CONDITIONS

The EA was tested on trending, ranging, and high-volatility environments. In all cases, the detection engine maintained stable behavior:

- In trend markets, the EA correctly identified sequences of unmitigated FVGs that formed along the trend direction.
- In ranging markets, the EA detected alternating bullish and bearish gaps, reflecting the oscillatory nature of price.
- During high-impact news events, the EA captured large FVGs created by sudden order-flow imbalances.

These results confirm that the detection logic is robust and adaptable to different market regimes. As illustrated in Fig. 4, the EA also highlights the confluence between Fibonacci retracement levels and Fair Value Gaps on a forex pair. The 61.8% retracement level frequently aligns with unmitigated FVGs, providing structural context for liquidity-based analysis.

C. MITIGATION PATTERNS

The mitigation module demonstrated consistent performance across all tested symbols. The EA correctly classified FVGs as mitigated only when a candle body overlapped the gap zone. Wick-only interactions were ignored, preventing false mitigation signals. Observed mitigation patterns included:

- full-body mitigation, where a candle closes entirely inside the gap,
- partial-body mitigation, where only part of the candle body overlaps the gap,
- delayed mitigation, where the price returns to the gap after several bars.

These behaviors align with institutional order-flow concepts and confirm the correctness of the implemented logic.

D. FIBONACCI CONTEXTUALIZATION

When the Fibonacci module was enabled, the EA provided additional context to interpret liquidity zones. The following observations were made:

- the 61.8% and 88.0% retracement levels frequently aligned with the origin of unmitigated FVGs,
- traders could visually identify the confluence between FVGs and demand or premium zones,
- the ruler updated dynamically as new swing highs and lows formed.

This confluence between Fibonacci retracement levels and Fair Value Gaps is illustrated in Fig. 4. This integration improved the interpretability of the chart and supported more informed decision-making.

E. CROSS-SYMBOL CONSISTENCY

The EA was deployed on a variety of instruments, including:

- EURUSD (forex),
- NASDAQ, SP 500, Russell 2000, DJIA (US indices),
- GOLD (commodity),

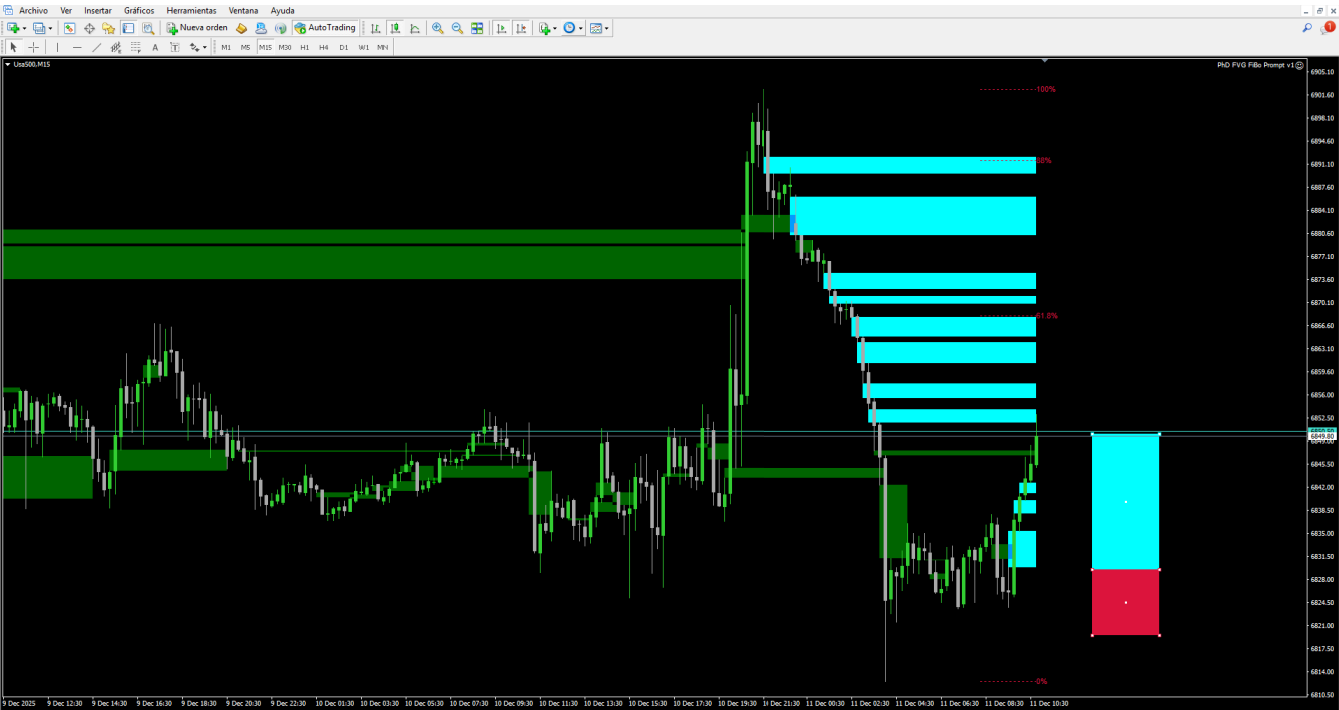


FIGURE 3. Detection of mitigated (green) and unmitigated (cyan) Fair Value Gaps at stock index. The EA dynamically updates the rectangles as new bars form.

- Eurostoxx50, IBEX 35, CAC 40, FTSE 100, DAX, FTSE MIB (European indices).

In all cases, the EA produced consistent and stable results. No symbol-specific adjustments were required, demonstrating the generality of the approach.

F. USER EXPERIENCE FEEDBACK

The informal feedback indicated that:

- the EA reduced the time required to manually scan charts for liquidity gaps,
- the color-coded visualization improved situational awareness,
- the combination of FVGs and Fibonacci levels provided a clear framework for identifying high-probability entry zones.

Overall, users reported increased confidence in their chart analysis and reduced cognitive load during live trading sessions.

VII. DISCUSSION

The results of this study demonstrate that generative AI can serve as an effective co-developer for domain-specific trading automation tools. However, several considerations and limitations must be acknowledged to contextualize the findings and outline future research directions.

A. MQL4-SPECIFIC HALLUCINATION PATTERNS

Although the AI-assisted workflow significantly reduced development time, Copilot exhibited hallucination patterns

specific to MQL4’s syntax and execution model—patterns qualitatively distinct from those reported in studies focused on general-purpose languages [2], [8]. MQL4 is a subset of C++ augmented with event-handling functions (OnTick, OnInit, OnDeinit) and platform-specific API calls that differ from standard C++ conventions. Two concrete classes of hallucinations were encountered:

1) UNDEFINED GLOBAL VARIABLE REFERENCES

In two instances, Copilot generated functions that referenced internal arrays (e.g., arrays for storing detected FVG boundaries and mitigation flags) as if they had been declared globally, without including the necessary global declarations at the file scope. In standard C++, such structures are often managed through class members or passed as references; in MQL4, they must be explicitly declared as global arrays before any function can access them. The compiler returned “undeclared identifier” errors for both cases, which were then fed back to Copilot as structured correction prompts, resolving the issue within a single additional cycle.

2) MITIGATION LOGIC CONFUSION (WICK VERSUS BODY OVERLAP)

The more persistent hallucination involved the mitigation condition. Copilot initially implemented a wick-based overlap check using Low[j] and High[j] comparisons—the most natural C-style test for price intersection—rather than the body-based check using Open[j] and Close[j] required by institutional order-flow principles. This required

three successive refinement cycles: in the first correction, Copilot introduced `Close[j]` but retained `High[j]` for one boundary; in the second, it correctly replaced both wick references but inverted the inequality for bearish gaps; in the third cycle, the fully correct body-based logic was produced. This pattern illustrates the challenge of encoding domain-specific semantics that have no direct parallel in standard programming practice.

These hallucinations are consistent with the broader finding by Barke et al. that LLM-generated code tends to be syntactically plausible but semantically misaligned with domain constraints [2]. The MQL4 context amplifies this effect because the language's deviation from standard C++ is subtle enough to produce code that compiles without errors yet behaves incorrectly.

B. BODY-BASED VERSUS WICK-BASED MITIGATION: RATIONALE AND IMPLICATIONS

The choice of body-based mitigation over the more common wick-based approach is a deliberate design decision grounded in institutional order-flow theory, and it represents one of the primary domain-specific requirements that the generative AI model had difficulty capturing without iterative guidance.

In standard technical analysis, a gap or price zone is considered “tested” when any part of a subsequent candle—including the wick—enters the zone. However, institutional order-flow frameworks distinguish between genuine price acceptance within a zone and transient price exploration. The candle body (open-to-close range) represents the interval during which the majority of transacted volume occurred; wicks, by contrast, represent brief and often low-volume excursions beyond the body. From this perspective, only a candle body that overlaps the FVG zone provides evidence that institutional orders were genuinely filled within the gap—an event that constitutes true mitigation of the liquidity imbalance.

The practical consequence of this choice is a significantly lower false-mitigation rate. Wick-based mitigation would classify a gap as resolved whenever price briefly touches the zone, even in high-volatility regimes where wicks frequently extend without sustained price acceptance. Body-based mitigation is therefore more conservative and more informative: an unmitigated FVG identified by the EA represents a price zone that has not yet attracted sufficient institutional participation to resolve the underlying order-flow imbalance.

The difficulty Copilot experienced in implementing this logic—defaulting to wick-based conditions across three refinement cycles—reflects the fact that body-based mitigation is a domain-specific semantic convention with no direct analog in mainstream programming paradigms. This case illustrates the importance of precise natural-language specification and iterative prompt refinement when encoding financial domain knowledge into AI-generated DSL code.

C. LLM SELECTION: COPILOT VERSUS ALTERNATIVES

This study employed Copilot as the sole generative AI tool. This choice was motivated by its IDE integration, inline suggestion capability, and partial training coverage of MQL4 and C++ code from public repositories. However, the study does not constitute a comparative evaluation of LLMs for MQL4 code generation.

Several alternative models would be relevant for comparison. GPT-4o, accessed through the OpenAI API or ChatGPT interface, supports longer context windows and richer multi-turn reasoning, which may reduce the number of refinement cycles required for complex domain-specific logic. Claude (Anthropic) is competitive in code-generation benchmarks and may handle nuanced natural-language specification more reliably. Specialized code models (e.g., Code Llama, StarCoder) might offer stronger syntactic coverage of C-derived DSLs but may lack the broad domain knowledge required to interpret financial specifications.

The question of whether LLM choice affects the number of refinement cycles—particularly for domain-specific logic such as body-based mitigation—is an important open question that this study cannot answer. Future work should conduct a systematic comparison of models under controlled conditions, using the same specification prompts and evaluating both cycle count and hallucination frequency.

D. GENERALIZATION ACROSS PLATFORMS

The proposed methodology was validated using MQL4 and the MetaTrader 4 platform. While the principles are transferable to other DSLs (e.g., Pine Script, EasyLanguage, MQL5), differences in syntax, execution models, and platform constraints may affect the generalizability of the results. Future work should evaluate the applicability of the prompting workflow across multiple trading platforms and DSL ecosystems.

E. INTERPRETABILITY AND HUMAN-IN-THE-LOOP REQUIREMENTS

Although the EA provides clear visualizations, the underlying logic—particularly mitigation rules and Fibonacci contextualization—still requires domain knowledge to interpret effectively. Traders must understand the meaning of FVGs, the implications of mitigation, the role of retracement levels, and the limitations of automated detection.

This reinforces the importance of maintaining a human-in-the-loop approach, where the EA acts as a decision-support tool rather than a fully autonomous trading system.

F. IMPACT OF MARKET MICROSTRUCTURE VARIABILITY

Liquidity behavior varies significantly across asset classes, brokers, and execution environments. Although the EA performed consistently across tested symbols, several factors may influence its behavior:

- differences in spread and execution latency,
- varying tick data quality,
- symbol-specific volatility patterns,
- broker-dependent candle formation.

These factors may affect the detection of FVGs or the timing of mitigation events. Additional testing on diverse data sources would strengthen the robustness of the approach.

G. ETHICAL AND PRACTICAL CONSIDERATIONS

The use of generative AI in financial engineering raises broader questions:

- **Transparency:** AI-generated code may obscure the origin of specific logic components.
- **Reproducibility:** Prompt-based workflows can be sensitive to small changes in wording, as the mitigation logic example in this paper illustrates.
- **Responsibility:** Traders must ensure that automated tools comply with regulatory and risk-management requirements.

These considerations highlight the need for responsible deployment practices and clear documentation of AI-assisted development processes.

H. FUTURE WORK

Several avenues for future research emerge from this study:

- Systematic comparison of multiple LLMs (GPT-4o, Claude, Code Llama) for MQL4 code generation under controlled conditions.
- Extension of the methodology to other DSLs and trading platforms (Pine Script, MQL5, EasyLanguage).
- Incorporation of automated unit testing for DSL code generated by AI.
- Exploration of reinforcement learning or hybrid AI models for adaptive gap detection.
- Evaluation of the EA on tick-level data to capture microstructure nuances.
- Statistical analysis of FVG detection performance over large-scale labeled datasets.
- Conducting a controlled experiment with human participants to provide stronger empirical evidence for the development-time efficiency gains reported in this study, enabling a rigorous comparison between GenAI-assisted and fully manual development workflows.

In summary, the findings of this study highlight both the potential and the current boundaries of generative AI in the development of domain-specific trading automation tools. The proposed workflow demonstrates clear engineering benefits—particularly in reducing development time and enabling iterative refinement—while the documented hallucination patterns underscore the continued necessity of domain-expert oversight. These considerations frame the broader context for the conclusions presented in the next section.

VIII. CONCLUSION

This paper presented a generative-AI-assisted methodology for developing domain-specific trading automation tools, with a focus on detecting and visualizing Fair Value Gaps (FVGs) in the MetaTrader 4 environment. Using Copilot to generate, refine, and debug MQL4 code through a structured prompting workflow, the proposed approach

reduced total development time to approximately four hours—a reduction of 65–73% compared to published manual development baselines for comparable DSL projects.

The resulting Expert Advisor integrates a robust FVG detection engine, a body-based mitigation module grounded in institutional order-flow principles, and a visualization layer that enhances chart interpretability. Optional Fibonacci contextualization further supports traders in identifying high-probability liquidity zones. Functional correctness was validated with precision and recall of 1.00 across 116 FVGs on seven instruments in an initial controlled evaluation, and confirmed by extended live deployment processing hundreds of candles daily over several months.

Beyond the implementation itself, the study characterizes two MQL4-specific hallucination patterns encountered during AI-assisted development—undefined global variable references and wick/body mitigation confusion—and documents the iterative prompt strategies that resolved them. These findings contribute a concrete example of how general-purpose LLMs must be guided through domain-specific semantic constraints to produce correct DSL code.

Future work may extend this approach to additional trading platforms and DSLs, incorporate automated testing frameworks for AI-generated code, and conduct a systematic comparison of LLMs under controlled conditions to evaluate the impact of model selection on refinement efficiency and hallucination frequency.

As generative AI continues to advance, its integration into financial engineering workflows is likely to become increasingly impactful, enabling faster development cycles and more accessible tooling for traders and developers alike.

APPENDIX PROMPTING WORKFLOW AND REUSABLE PROMPT TEMPLATES

This appendix formalizes the prompting strategy used during the development of the Expert Advisor (EA). Table 4 presents the core prompt templates as reusable patterns applicable to other DSL development contexts. Each template is structured with a *trigger condition* (when to use it), a *pattern* (the natural-language structure of the prompt), and an *example instantiation* from this study.

A. INITIAL SPECIFICATION PROMPT

The development process began with a high-level natural-language specification describing the intended behavior of the EA (see the *Initial Specification* template in Table 4). This prompt established the functional scope and served as the foundation for the initial code generation.

B. COMPILER-GUIDED REFINEMENT PROMPTS

During iterative development, compilation errors were fed back to the model using structured prompts (the *Compilation Error Correction* template). This approach enabled rapid resolution of syntactic and semantic issues. In this study, two undefined-variable errors were corrected in a single cycle using this template.

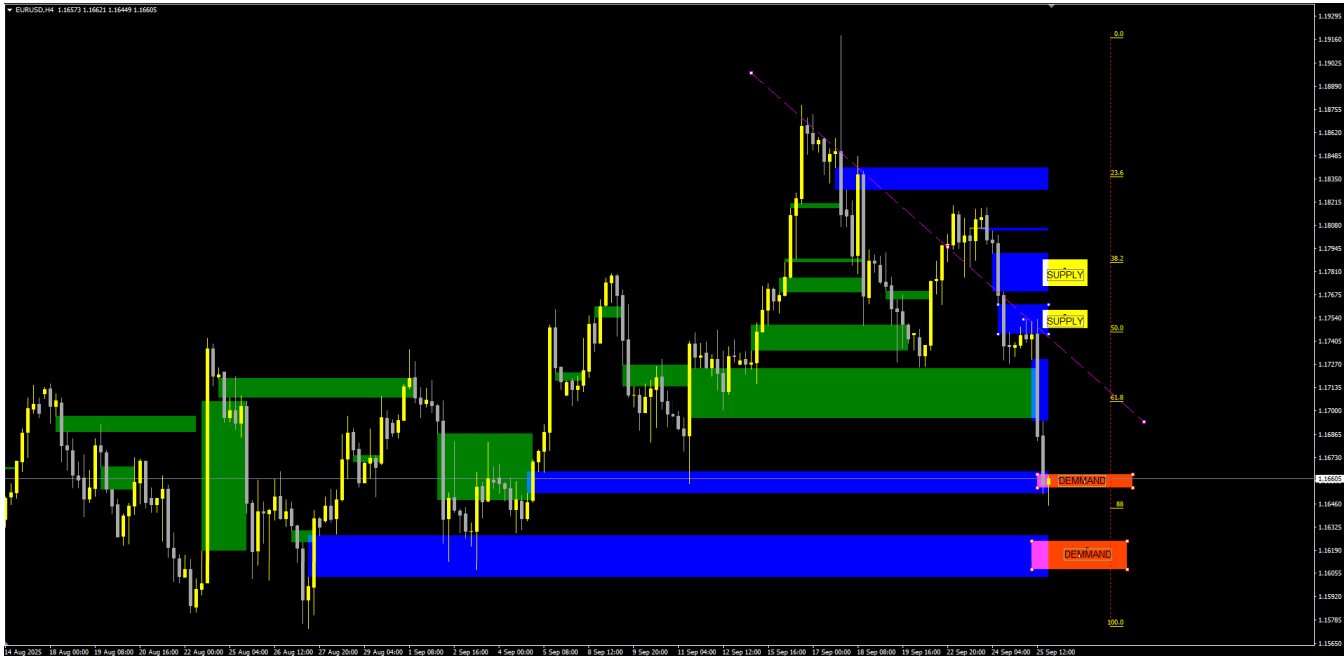


FIGURE 4. Confluence between Fibonacci retracement levels and Fair Value Gaps on a forex pair. The 61.8% and 88.0% levels align with the origin of an unmitigated FVG, providing structural context for liquidity-based analysis.

TABLE 4. Reusable prompt templates for AI-assisted DSL development.

Template Name	Prompt Pattern	Example (MQL4 / this study)
Initial Specification	“Develop a {DSL} program that {high-level behavior}. Include {constraints}. Use {platform-specific conventions}.”	“Develop an MQL4 Expert Advisor that detects bullish and bearish FVGs using a three-candle pattern. Classify each FVG as mitigated or unmitigated based on candle-body overlap. Draw color-coded rectangles and update them dynamically.”
Compilation Error Correction	“The following {DSL} compilation errors occurred. Correct the code while preserving the intended logic: {error list}. Provide only the corrected code block.”	“Undeclared identifier ‘fvgUpper[]’ in function Detect-FVG(). Undeclared identifier ‘mitStatus[]’. Declare these as global arrays with appropriate types.”
Logic Correction	“The current implementation {describes incorrect behavior}. Update the logic so that {correct behavior}. {domain constraint} must be respected.”	“The EA marks FVGs as mitigated when only the wick overlaps. Update the mitigation condition to use Open and Close values only. Wick interactions must be ignored.”
Visualization Fix	“{Graphical element} is not behaving as expected: {description of defect}. Modify the {drawing/update} logic so that {expected behavior}.”	“Rectangles for unmitigated FVGs are not extending to the current bar. Modify the right-boundary update so it advances with each new tick.”
Feature Addition	“Add {feature description} to the existing implementation. The feature should {behavior specification}. Expose a user-configurable {parameter} to enable or disable it.”	“Add a Fibonacci retracement ruler using the most recent swing high and low. Include levels 0%, 61.8%, 88.0%, and 100%. Allow the user to enable or disable via an input parameter.”

C. LOGIC CORRECTION PROMPTS

When functional inconsistencies were observed during chart testing, targeted prompts using the *Logic Correction* template were applied. For the mitigation logic, four successive applications of this template were required before the correct body-based implementation was produced.

D. VISUALIZATION REFINEMENT PROMPTS

Graphical issues were addressed through *Visualization Fix* template prompts. The right-boundary extension defect was resolved within a single cycle.

E. FIBONACCI MODULE PROMPTS

The optional Fibonacci contextualization was added using the *Feature Addition* template. Two refinement cycles were required to converge on a stable implementation with correctly positioned levels and responsive updates.

APPENDIX
PSEUDO-CODE OF THE DETECTION AND MITIGATION ENGINE

This appendix presents a mathematical formulation of the core logic implemented in the Expert Advisor. The notation

abstracts away MQL4-specific syntax while preserving the algorithmic structure.

F. FAIR VALUE GAP DETECTION

For each index $i \in \{0, \dots, N - 3\}$, define:

$$\begin{aligned} h_0 &= \text{High}[i], \\ l_0 &= \text{Low}[i], \\ h_2 &= \text{High}[i + 2], \\ l_2 &= \text{Low}[i + 2]. \end{aligned}$$

A bullish FVG exists if:

$$l_2 > h_0,$$

with:

$$\text{upper} = l_2, \quad \text{lower} = h_0, \quad \text{originIndex} = i.$$

A bearish FVG exists if:

$$h_2 < l_0,$$

with:

$$\text{upper} = l_0, \quad \text{lower} = h_2, \quad \text{originIndex} = i.$$

G. MITIGATION LOGIC

For each FVG with bounds (lower, upper) and origin index k , mitigation is evaluated over:

$$j \in \{k + 3, \dots, \text{currentBar}\}.$$

Define the candle body bounds at bar j as:

$$b_{\min}(j) = \min(O_j, C_j), \quad b_{\max}(j) = \max(O_j, C_j).$$

For a **bullish** FVG, body-based mitigation occurs at the first j satisfying:

$$b_{\max}(j) < \text{upper} \quad \text{and} \quad b_{\min}(j) > \text{lower}.$$

For a **bearish** FVG, mitigation occurs at the first j satisfying:

$$b_{\min}(j) > \text{lower} \quad \text{and} \quad b_{\max}(j) < \text{upper}.$$

Wick-based overlap ($\text{Low}[j] \leq \text{upper}$ or $\text{High}[j] \geq \text{lower}$) is explicitly excluded from the mitigation condition, ensuring that only genuine body overlap is recorded as mitigation.

REFERENCES

- [1] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proc. CHI Conf. Human Factors Comput. Syst. Extended Abstr.*, Apr. 2022, pp. 1–7.
- [2] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, pp. 85–111, Apr. 2023.
- [3] S. MacNeil, A. Tran, A. Hellas, J. Kim, S. Sarsa, P. Denny, S. Bernstein, and J. Leinonen, "Experiences from using code explanations generated by large language models in a web software development course," in *Proc. 54th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE)*, Mar. 2023, pp. 931–937.
- [4] J. Hasbrouck and G. Saar, "Low-latency trading," *J. Financial Markets*, vol. 16, no. 4, pp. 646–679, Nov. 2013.
- [5] T. Foucault, M. Pagano, and A. Röell, *Market Liquidity: Theory, Evidence, and Policy*. London, U.K.: Oxford Univ. Press, 2013.
- [6] M. O'Hara, "High frequency market microstructure," *J. Financial Econ.*, vol. 116, no. 2, pp. 257–270, May 2015.
- [7] E. Kant, "Domain Specific Languages and the Acceleration of Computational Finance," in *Proc. 4th Workshop on High Perform. Comput. Finance (WHPCF '11)*, New York, NY, USA: ACM, Nov. 2011, pp. 1–2, doi: 10.1145/2088256.2088258.
- [8] Sobania et al., "An analysis of the automatic bug fixing performance of ChatGPT," in *Proc. IEEE/ACM Int. Workshop Automated Program Repair (APR)*, May 2023, pp. 23–30.
- [9] Venkadesan et al., "Domain-specific generative AI models for advancing financial services," *TechRxiv Preprint*, Jan. 2025, doi: 10.36227/techrxiv.176184757.79808289/v1.
- [10] M. N. Ugural, M. R. Ozyilmaz, and H. I. Burgan, "Life cycle assessment analysis based on material selection in sustainable airport buildings," *Buildings*, vol. 14, no. 9, p. 2728, Aug. 2024.
- [11] Y. Hwang, Y. Lee, J. Lee, S. Zohren, J. H. Kim, W. C. Kim, Y. Lee, and F. J. Fabozzi, "Deep learning in asset management: Architectures, applications, and challenges," *J. Portfolio Manage.*, vol. 52, no. 2, pp. 233–271, Oct. 2025. [Online]. Available: <https://ssrn.com/abstract=5593850>

IGNACIO FUMANAL-ANDRÉS received the M.S. degree in software engineering research from the National University of Distance Education (UNED), Madrid, Spain, in 2021, where he is currently pursuing the Ph.D. degree in engineering systems and control with the Computer Science College. He has extensive professional experience in software and telecommunications engineering, having worked for major international vendors, such as Fujitsu Limited Japan, Bay Networks, Nortel Networks, Riverstone Networks, RAD Data, Huawei Technologies, ZTE Corporation, and Juniper Networks/HPE. His background includes the development of banking application software using the APL*Plus DSL, low-level ATM programming in Fujitsu's Panafacom line of 8-bit and 16-bit microprocessor assembly language, and McDonnell Douglas's PRO-IV 4GL low-code platform development during product-marketing responsibilities across Southeast Asia, while working for Fujitsu Limited Japan. For the applicability demonstration of his Master's thesis—focused on fuzzy-logic-based trading-strategy coding, he employed both Java and MQL4. His current research interests include software engineering applied to the automation and processing of trading platforms and algorithmic trading systems. His area of research interests include low-code and no-code development platforms, as well as the use of generative AI for code co-creation in financial index and currency markets.

...